2026 23rd International Joint Conference on Computer Science and Software Engineering (JCSSE) | 979-8-3315-8200-5/26/$31.00 ©2026 IEEE | DOI: 10.1109/JCSSE68839.2026.11596610

Vision-based Hand Gesture Virtual Keyboard-Mouse framework with Bilingual Next-word prediction

Sammam Mahdi
Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
sammam.mahdi@gmail.com

Reshad Ul Karim
Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
reshad.sazid@gmail.com

Tareq Sujat
Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
sujattareq@gmail.com

Syeda Maliha Tabassum
Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
syedamalihatabassum19@gmail.com

Abrar Samin
Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
abrarsamin100@gmail.com

Aniqua Nusrat Zereen
Faculty of Information and Communication Technology
Mahidol University
Salaya, Thailand
aniqua.zer@mahidol.ac.th

***Abstract*—Touchless human–computer interaction (HCI) is increasingly sought for improved accessibility, yet existing gesture-driven keyboards remain hindered by limited vocabularies, sensitivity to illumination, high computational overhead, and inadequate support for multiple languages. To address these challenges, a modular, non-probabilistic pipeline is proposed that integrates real-time geometric encoding of MediaPipe hand landmarks for gesture recognition and two-layer long-short-term memory (LSTM) networks for multilingual next-word prediction in English and Bangla. A unified keyboard-mouse interface maps recognized gestures to on-screen key and pointer actions, while a lightweight performance monitor logs frame rate, latency, and resource usage. All components run sequentially in real time. Extensive evaluation on various hardware platforms demonstrates that nine static gestures are classified with at least 97.4% accuracy and under 0.03 ms per gesture latency, allowing frame rates up to 37.7 FPS. LSTM models achieve accuracies above 97% and perplexities below 1.15, resulting in character error rates of 3.43% for English and 8.34% for Bangla. Cross-platform statistical analysis confirms consistent usability and resource efficiency. The proposed system thus offers a low-latency scalable solution for multilingual and touchless text entry on desktop platforms, with potential extension to wearable devices in future work.**

***Index Terms*—Gesture Recognition, Virtual Keyboard, Touchless Text Input, Word Prediction**

I. Introduction and Background

Touchless human-computer interaction is gaining interest in applications such as AR/VR and wearable computing. Existing gesture-driven keyboards, however, remain constrained by limited gesture vocabularies, lighting sensitivity, high computational costs, and poor support for diverse scripts. Touchless HCI is shown to enhance device accessibility, hygiene, and usability by enabling contactless mid-air typing and pointing. Multimodal systems combining EyeTribe gaze dwell and Myo-armband motions achieve 8.8 Letters Per Minute (LPM) at 57 bps but are hampered by dwell thresholds and limited tracking resolution [1]. A TensorFlow-based webcam keyboard detects ten regions at 96% accuracy yet remains vulnerable to lighting changes and data scarcity [2]. Continuous in-air character scripting using hue, saturation, and value (HSV) flick motions yields 98% accuracy but incurs two-second keystroke delays and supports only seven gestures [3]. A wavelet-support vector machine (SVM) approach with Myo signals and camera tracking reaches 97% accuracy at 45 clicks/minute yet is constrained by hardware wearability [4]. MediaPipe landmark–driven mouse control achieves 96% success and 30 ms latency but suffers from illumination sensitivity and a five-gesture limit [5]. Lightweight Convolutional Neural Networks (CNN) trained on MediaPipe landmarks permit static gesture typing but exhibit low robustness under poor lighting and single-hand testing [6]. Temporal modeling has further advanced recognition. Leap Motion with recurrent neural network (RNN) distinguishes six wrist positions at 99% accuracy but restricts typing speed to 11 Letters Per Minute (LPM) [7]. A CNN-LSTM based system using MediaPipe-Holistic landmarks attains 98.5% accuracy for ten gestures, yet faces lighting and computational challenges [8]. A gesture-based multilingual keyboard using CronoNet and MediaPipe landmarks enables English–Bangla typing with over 96% accuracy, though it lacks real-time evaluation and diverse user testing [9]. Recent prototypes integrate typing and pointing: MediaPipe-PyAutoGUI stacks provide configurable gesture control but lack comprehensive performance metrics [10] [11]. An AR air-tapping keyboard using Leap Motion reduces errors by 27% at 7.5 Words Per Minute (WPM) across ten users but remains limited by low speed and generalization [12]. A two-stream CNN trained on top and side MediaPipe views is proposed, achieving 94.3% accuracy, though performance drops in cluttered scenes [13]. Similarly, an egocentric Gated Recurrent Unit (GRU) -based gesture keyboard reaches 96.2% accuracy using hand pose landmarks that seamlessly switches between English and Bangla layouts but remains sensitive to lighting and hand

TABLE I
COMPARISON OF REPRESENTATIVE PRIOR GESTURE-BASED SYSTEMS

Ref	System	Accuracy	Speed / Latency	Key Limitation	Proposed System
[2]	TF Webcam Classifier	96%	—	Lighting sensitivity; limited data	Geometric encoding; robust across 7 lighting conditions
[3]	HSV Flick Scripting	98%	2 s/key	2 s delay; only 7 gestures	$<$0.03 ms latency; 9-gesture vocabulary
[4]	Wavelet-SVM + Myo	97%	45 click/min	Wearable hardware required	Camera-only; no specialized hardware
[5]	MediaPipe Mouse	96%	30 ms	5 gestures only; no keyboard	9 gestures; full keyboard + mouse
[6]	Lightweight CNN	$\sim$96%	Moderate	Poor lighting robustness; single-hand	Validated under 7 lighting conditions
[7]	Leap Motion + RNN	99%	11 LPM	Depth hardware; low throughput	USB webcam; up to 37.7 FPS
[8]	CNN–LSTM + MediaPipe	98.5%	Moderate	High compute; lighting issues	Lightweight geometric engine; low CPU
[9]	CronoNet (EN+BN)	$>$96%	—	No real-time eval; no cross-platform	Real-time; 5-platform t-test
[13]	Two-Stream CNN	94.3%	Moderate	Cluttered scene degradation; high compute	Non-probabilistic; stable across scenes
[14]	Egocentric-GRU (EN+BN)	96.2%	Moderate	Lighting sensitive; no NWP; no cross-platform	7-condition test; bilingual LSTM NWP

variation [14]. Despite high recognition rates, remaining challenges include gesture diversity, illumination robustness, and end-user usability. Table I compares representative prior works with reported accuracies against the proposed system.

To address these challenges, a non-probabilistic pipeline is proposed that combines real-time geometric encoding of MediaPipe [15] landmarks for gesture recognition, multilingual Long Short-Term Memory (LSTM) -based next-word prediction in English and Bangla, and a low-latency interface, demonstrating robust performance across multiple hardware platforms. The key contributions of the proposed study:

- **Dual-Mode Interaction:** QWERTY (English) and phonetic (Bangla) on-screen keyboards and virtual mouse controls (cursor movement, clicks, drag and drop) allow smooth text entry and pointing using the same gesture vocabulary.
- **Multilingual Next-Word Prediction:** Custom two-layer LSTM models are trained for English and Bangla, achieving top-1 accuracies above 97% and perplexities below 1.15, and are integrated into the gesture interface to provide real-time, top-K suggestions.
- **Cross-Platform Validation:** A thorough evaluation on five hardware platforms shows sustained real-time performance (up to 37.7 FPS), low character error rates (3.43% for English, 8.34% for Bangla), and considerable improvements in latency and throughput.

These results show that scalable, touchless virtual keyboards are feasible on the evaluated desktop-class platforms with potential extension to stationary and wearable systems in future work. This enables practical, sanitary, and accessible text entry across languages and device classes.

II. METHODOLOGY

The proposed modular pipeline combines real-time gesture recognition, LSTM-based multilingual next-word prediction, and low-latency user interface (UI) rendering to deliver robust touch-less text entry with comprehensive performance evaluation across diverse hardware.

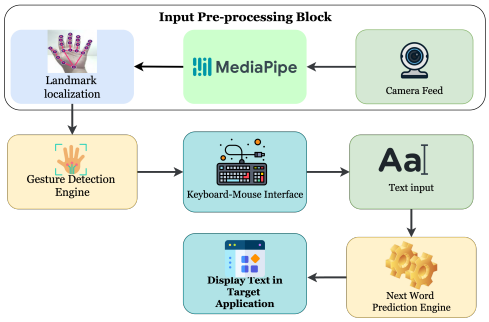


Fig. 1. Modular pipeline of virtual keyboard-mouse application.

A. System Architecture Overview

A modular pipeline is adopted to separate input processing, gesture recognition, language modeling, UI interaction, and performance monitoring. The system starts with a camera feed, processed via MediaPipe for real-time landmark localization. These landmarks are passed to a non-probabilistic Gesture Detection Engine for gesture interpretation. Detected gestures are mapped through a unified Keyboard-Mouse Interface for on-screen input. Text is enhanced by a Next Word Prediction Engine and displayed in the target application. A lightweight Performance Monitor logs frame rate, latency, and system usage. All modules run sequentially in a real-time loop. The pipeline is illustrated in Fig. 1.

B. Data Acquisition and Pre-processing

1) Gesture Data Collection: Real-time hand landmark data are captured using a USB webcam. The hand-tracking model in MediaPipe employs detection and tracking confidence criteria to facilitate precise landmark extraction. This initial calibration mechanism normalizes hand positioning by using a standardized coordinate system relative to the detected hand region. It scales and translates the landmarks so that the hand's size and position are consistent across frames, regardless of distance from the camera or hand orientation. This normalization helps adjust to variations in lighting and different hand sizes and positions, enabling stable and accurate landmark extraction in real time.

2) Text Corpus Assembly: English corpora are sourced from Project Gutenberg [16] and Bangla corpora from Kaggle [17]. All texts are cleaned, special characters, punctuation marks, and extra spaces are removed, and English texts are lowercased. Tokenization is performed using language-specific rules that split text into words. Vocabularies are limited to a fixed size, using `<PAD>` for sequence padding and `<UNK>` to represent unknown or rare words not seen during training.

C. Gesture Detection Engine

This module implements non-probabilistic hand gesture recognition and maps recognized gestures to keyboard and mouse actions.

1) Hand Gesture Recognition: Hand landmarks are extracted from real-time frames via MediaPipe, yielding 21 (x, y) points per hand. Finger extension is determined by the ratio of Euclidean distances between fingertip/base and base/palm as described in (1). The four binary extension states (index, middle, ring, pinky) are encoded into a 4-bit integer as summarized in Algorithm 1 where d_1 and d_2 denote the fingertip–base and base–palm distances, respectively.

$$f(x) = \begin{cases} 1, & \dfrac{d_1}{d_2} > 0.5, \\ 0, & \text{otherwise} \end{cases} \tag{1}$$

Algorithm 1 Finger State Encoding

$S \leftarrow 0$
for each finger in [index, middle, ring, pinky] **do**
  compute d_1, d_2
  if $d_1/d_2 > 0.5$ **then**
    $S \leftarrow (S \ll 1) \vee 1$
  else
    $S \leftarrow S \ll 1$
  end if
end for
return S

2) Keyboard–Mouse Interaction: Recognized gestures drive two interaction modes.

a) Virtual Mouse: Cursor movement is enabled by a sustained `v` gesture over two frames. The middle-finger base serves as reference, and dampening filters smooth cursor trajectories. Mouse clicks, double-clicks, and drag-and-drop are mapped to `left_click`, `right_click`, `index_mid_closed`, and the `grab`→`open_hand` sequence, respectively. Gesture states for keyboard-mouse interactions are shown in Fig. 2.

b) Virtual Keyboard: The "pinky-down" gesture is implemented to register input clicks on the keyboard. The gesture is detected whenever the pinky fingertip (MediaPipe $landmarks\_20$) lies below its proximal interphalangeal joint ($landmark\_18$) in image coordinates. To avoid multiple triggers while the finger remains down, a simple debounce flag is maintained as shown in Algorithm 2.

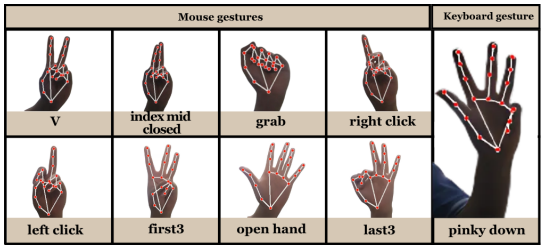


Fig. 2. Gesture operations of virtual keyboard-mouse interactions.

Algorithm 2 Pinky Down Gesture Detection (with debounce)

Initialize $pinky\_prev\_state \leftarrow$ `False`
while receiving video frames **do**
  if hand landmarks are detected **then**
    for each hand in detected hands **do**
      $pinky\_tip\_y \leftarrow$ Y-coordinate of landmark 20
      $pinky\_pip\_y \leftarrow$ Y-coordinate of landmark 18
      $pinky\_down \leftarrow pinky\_tip\_y > pinky\_pip\_y$
      if $pinky\_down$ **and** NOT $pinky\_prev\_state$ **then**
        $pinky\_prev\_state \leftarrow True$
        $TriggerAction$
      else if NOT $pinky\_down$ **and** $pinky\_prev\_state$ **then**
        $pinky\_prev\_state \leftarrow False$
      end if
    end for
  end if
end while

D. Language Model Training

A two-layer LSTM (128 units per layer, 64-dim embedding) with dropout (0.5) and batch normalization is trained using Adam ($\alpha = 10^{-3}$, $L_2 = 10^{-5}$) for up to 100 epochs with early stopping (patience=5). A fixed vocabulary of 2,000 tokens (with `<PAD>`, `<UNK>`) is used, and automatic language detection switches between English and Bangla tokenization. The 10,000-word dataset for both English and Bangla is divided into training and validation sets in an 80:20 ratio. Context vectors are fed to the LSTM to produce logits which are softmax-normalized and filtered for top-K suggestions. Gesture-to-action mapping inserts characters or cursor events, and predictions are rendered as on-screen buttons. Automatic language detection is based on Unicode vs. ASCII text counts.

E. Keyboard Interface and Rendering

1) Layout Definition: English QWERTY and Bangla phonetic keyboards are defined as two-dimensional arrays of key labels, including special keys (`lang`, `pred-1`, `pred-2`, `pred-3`, `Color`). Given key width w, height h, horizontal spacing s, and N keys per row, the total keyboard width and horizontal offset on a 1280×720 canvas ($W_{\text{can}} = 1280$) are computed. Vertical centering is performed analogously. The virtual keyboard interface for English and Bangla keyboard is shown in Fig. 3 and Fig. 4.

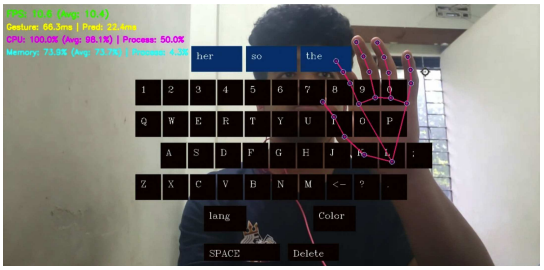


Fig. 3. English Language Virtual keyboard interface.

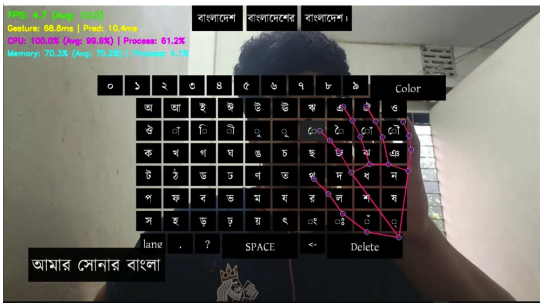


Fig. 4. Bangla Language Virtual keyboard interface.

2) Text Rendering Techniques: English characters are rendered via OpenCV's text functions, while Bangla glyphs are shaped by `HarfBuzz` [18] and drawn using `PyQt5`. This ensures accurate, high-fidelity rendering across both language.

3) Interactive Feedback: Key hover states are indicated by applying a contrasting fill when the index-tip projection enters a key's ROI. Selections (pinky-down gesture) are highlighted with a filled rectangle.

F. Performance Metrics Collection and Analysis

The frame rate at each iteration is estimated based on the inverse of the time difference between consecutive frame timestamps. To mitigate short-term fluctuations and provide a more stable performance indicator, a rolling average is computed over the most recent M frames. This approach allows for monitoring both instantaneous and smoothed trends in system responsiveness. Gesture and prediction latencies are measured via high-resolution timers. CPU and memory usage are sampled at 1 Hz in a background thread; samples are buffered and flushed to session logs every 30 frames. Logged arrays undergo descriptive statistical analysis.

G. Experimental Setup and Evaluation Protocol

Experiments are conducted on five platforms: AMD Ryzen 6700H, Apple MacBook Pro M2 , Intel Core i5-12400 (12th Gen), Intel Core i5-4460 (4th Gen), and AMD Ryzen 5 5600G. Metrics include cross-entropy loss, perplexity, token and top-K accuracy, FPS, gesture/prediction/end-to-end latency, CPU/memory usage, and character error rate (CER). CER is defined as shown in (2).

$$\mathrm{CER}_{\%} = \left(\frac{S + D + I}{N}\right) \times 100\% \tag{2}$$

Here S, D, and I denote substitutions, deletions, and insertions, respectively, and N is the total reference characters.

TABLE II
HAND GESTURE RECOGNITION ACCURACY

Gesture	Frames	Correctly Classified	Misclassified	Accuracy (%)
index mid closed	300	295	5	98.3%
v	300	298	2	99.3%
left click	300	292	8	97.3%
right click	300	293	7	97.7%
grab	300	300	0	100.0%
open hand	300	300	0	100.0%
first3	300	297	3	99.0%
last3	300	294	6	98.0%
pinky down	300	299	1	99.7%

TABLE III
RECOGNITION ACCURACY OF CHARACTER INPUT

Input Type	No. of Characters	Accurate Input	Input Duration (s)	Input Accuracy (%)
Random Character (Index)	85	82	60s	96.5%
Same Character (Index)	90	84	60s	93.3%
Random Character (pinky down)	85	82	60s	96.5%
Same Character (pinky down)	90	86	60s	95.6%

TABLE IV
HAND GESTURE RECOGNITION

Gesture Name	Recognition Time (ms)
index mid closed	0.0256
v	0.0166
left click	0.0048
right click	0.0047
grab	0.0049
open hand	0.0056
first3	0.0053
last3	0.0040
pinky down	0.0042

Statistical significance is assessed via paired t-tests as shown in (3) with $\alpha = 0.05$.

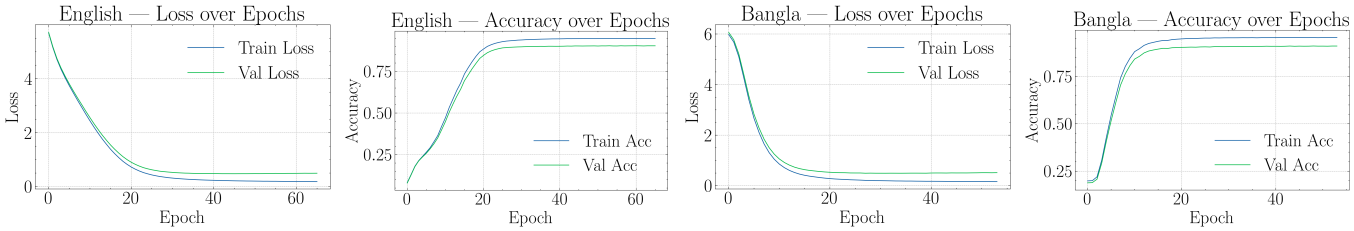
$$t = \frac{\bar{d}}{s_d/\sqrt{n}} \tag{3}$$

The mean difference is denoted as $\bar{d}$, the sample standard deviation as s_d, and there are n pairs. To address illumination sensitivity and evaluate real-world robustness, additional experiments were conducted under seven distinct lighting conditions, including bright natural light (2000 lx), normal indoor light (500 lx), warm and cool artificial lighting, backlit scenarios, directional shadow conditions, dynamic/flickering light, and dim light (50 lx). Evaluations were performed on three participants spanning Fitzpatrick skin types II, IV, and V, across multiple hand orientations. Metrics recorded include gesture recognition accuracy, false negative rate (FNR), and response latency.

III. RESULTS

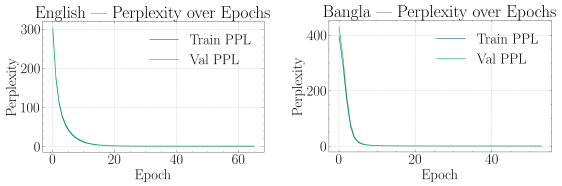
A. Gesture Result Evaluation

The frame-level classification results in Table II demonstrate that all nine gestures are recognized with at least 97.3% accuracy, with the 'grab' and 'open hand' gestures achieving perfect (100%) recognition. Character-level input accuracy shown in Table III highlights improved performance of the pinky down gesture over the previous index-based system [14], with accuracy increasing from 93.3% to 95.6% in repeated input scenarios. Summarised hand gesture recognition time is shown in Table IV.



(a) (b) (c) (d)

Fig. 5. Training curves for English and Bangla LSTM models: (a) English model loss, (b) English model accuracy, (c) Bangla model loss, (d) Bangla model accuracy.



(a) (b)

Fig. 6. Train and Validation Perplexity (PPL) over Epoch curves for the (a) English and (b) Bangla LSTM models.

B. Language Model Evaluation

a) English Language Model: Fig. 5a illustrates that the cross-entropy loss for both training and validation decreases significantly from 5.8 to 0.14 across 60 epochs, indicating negligible overfitting. The token-level accuracy rises from below 10% to approximately 97%, as illustrated in Fig. 5b. Figure 6a illustrates a reduction in perplexity from 300 to 1.15. The English model attains a final cross-entropy loss of 0.1365, a perplexity of 1.1463, a top-1 accuracy of 97.00%, a top-3 accuracy of 99.11%, and a top-5 accuracy of 99.61% on the hold-out test set.

b) Bangla Language Model: A similar trend is observed for the Bangla language model as shown in Fig. 5c. Training and validation losses descend from about 6.0 to 0.095 while accuracy climbs to 97.16% (Fig. 5d) and perplexity drops from approximately 420 to 1.10 as demonstrated by Fig. 6b. Quantitatively, the Bangla model records a test cross-entropy loss of 0.0954, perplexity of 1.1001, top-1 accuracy of 97.16%, top-3 accuracy of 99.26%, and top-5 accuracy of 99.56%.

C. Pairwise Statistical Analysis of Device Performance

Pairwise *t*-tests are conducted to evaluate whether performance metrics significantly differ across devices. The *t*-test assesses whether the means of two groups are statistically distinct, and the resulting *p*-value indicates the probability of observing such a difference by chance. A threshold of $p < 0.05$ was used to determine statistical significance. As shown in Table V, most inter-device comparisons exhibit significant differences in FPS, gesture latency, total latency, and memory usage, highlighting the strong influence of hardware on performance. In contrast, CPU usage does not vary significantly in some comparisons among mid-range devices. This suggests possible saturation in computational demand where processing capacity may no longer be the performance bottleneck.

D. Device Performance Comparison

Table VI illustrates the averages for FPS, latency, CPU, and memory utilization across five platforms. Optimal performance is attained on the Ryzen 5 5600G and MacBook Pro M2; Intel Core i5 12th and 4th-generation devices exhibit reduced FPS and increased latency, with the latter utilizing the majority of CPU resources. The additional memory on M2 is due to operating system behavior, highlighting the necessity for multi-device benchmarking.

E. Error-Rate Analysis

The CERs at sentence level for English range from 0.68% to 7.25%, with an average of 3.43% (σ = 1.68%) as shown in Table VII. In Bangla, CERs vary between 4.85% and 12.82%, averaging 8.34% (σ = 2.96%). These low error rates indicate that the virtual keyboard system delivers highly accurate character-level transcription in both languages. English text achieves very tight error limits, while Bangla, despite its more complex language, still maintains the CER 13% in all test sentences.

F. Lighting Robustness Evaluation

The proposed system was evaluated across seven lighting conditions (50 lx to 2000 lx), achieving accuracies between 91.7% and 98.3% with a maximum degradation of only 6.6 percentage points, while response latency remained stable between 108 ms and 136 ms. Even under adverse conditions such as dim or dynamic illumination, accuracy stayed above 90% with graceful degradation, confirming real-time usability. These results validate the system's robustness and reliability under realistic lighting variability as shown in Table VIII.

IV. Discussion and Conclusion

The proposed system is observed to achieve exceptional gesture recognition accuracy (>=97.4%) and ultra-low latency (< 0.03 ms), thus providing a highly responsive foundation for touchless text entry. Versatility across diverse hardware is demonstrated through a modular design and cross-platform evaluation. Multilingual capability is highlighted by support for both English and Bangla language. The present nine-gesture vocabulary provides a steady foundation for dynamic and two-hand interactions.The

TABLE V
PAIRWISE T-TEST P-VALUES FOR ALL METRICS BETWEEN DEVICES.

Device 1	Device 2	FPS	Gesture Latency	Total Latency	CPU	Memory
Ryzen 7 6800H 16GB	MacBook Pro M2 16GB	**6.313e-247**	**1.217e-84**	**4.519e-43**	**4.011e-44**	**0**
Ryzen 7 6800H 16GB	i5 12th gen 16GB	**0.0036**	**2.186e-06**	**3.209e-06**	0.05301	**2.578e-249**
Ryzen 7 6800H 16GB	Ryzen 5 5600G 16GB	**0**	**4.728e-84**	**1.011e-53**	0.3088	**6.475e-288**
Ryzen 7 6800H 16GB	i5 4th gen 16GB	**1.444e-47**	**9.6e-70**	**2.864e-23**	**1.02e-284**	**6.769e-217**
MacBook Pro M2 16GB	i5 12th gen 16GB	**3.132e-258**	**4.486e-62**	**8.752e-68**	**3.049e-15**	**0**
MacBook Pro M2 16GB	Ryzen 5 5600G 16GB	**7.779e-92**	**7.227e-38**	**5.721e-298**	**5.056e-174**	**0**
MacBook Pro M2 16GB	i5 4th gen 16GB	**2.317e-199**	**0**	**2.983e-136**	**0**	**0**
i5 12th gen 16GB	Ryzen 5 5600G 16GB	**0**	**1.273e-60**	**4.255e-76**	0.1213	**6.206e-191**
i5 12th gen 16GB	i5 4th gen 16GB	**8.237e-59**	**8.285e-44**	**7.638e-57**	**9.507e-120**	**2.316e-166**
Ryzen 5 5600G 16GB	i5 4th gen 16GB	**0**	**0**	**2.208e-256**	**0**	**4.879e-131**

TABLE VI
PERFORMANCE OVERVIEW OF THE SYSTEM ACROSS DEVICES

Device	Avg FPS	Gesture Latency (s)	Total Latency (s)	Avg CPU (%)	Avg Memory (%)
i5 4th gen 16GB	13.66	0.0467	0.0739	93.31%	57.05%
Ryzen 7 6800H 16GB	10.33	0.0898	0.1044	22.28%	47.90%
i5 12th gen 16GB	9.75	0.0814	0.1177	24.24%	62.68%
MacBook Pro M2 16GB	28.24	0.0220	0.0489	32.75%	75.05%
Ryzen 5 5600G 16GB	37.70	0.0250	0.0311	22.81%	58.00%

TABLE VII
SUMMARY OF CHARACTER ERROR RATE (CER) STATISTICS

Language	Min CER (%)	Max CER (%)	Mean $\pm$ Std (%)
English	0.68	7.25	3.43 ± 1.68
Bangla	4.85	12.82	8.34 ± 2.96

TABLE VIII
GESTURE RECOGNITION PERFORMANCE ACROSS LIGHTING CONDITIONS

Lighting Condition	Lux (lx)	Avg. Accuracy (%)	Avg. FPR (%)	Avg. FNR (%)	Avg. RT (ms)
Normal Indoor Light	500	98.29	1.00	1.71	108.57
Cool Artificial Light	600	97.83	1.17	2.17	108.67
Bright Natural Light	2000	97.57	1.43	2.43	113.57
Warm Artificial Light	300	95.83	2.17	4.17	122.33
One-Sided Shadow Light	200	94.17	3.17	5.83	126.67
Backlit Condition	800	93.17	4.50	6.83	132.17
Dim Light	50	92.50	3.83	7.50	136.50
Dynamic/Flickering Light	150	92.17	4.83	7.83	135.83

proposed system demonstrates validated robustness to illumination variability, maintaining over 90% accuracy across seven diverse lighting conditions, including low-light and dynamic illumination scenarios. While performance degrades slightly under extreme conditions, the degradation is gradual and does not compromise usability.Future enhancements include adding dynamic and bimanual gestures, incorporating adaptive lighting compensation and background-invariant preprocessing, and exploring lightweight transformer architectures for multilingual next-word prediction on edge devices with reduced latency. To validate real-world applicability, AR/VR headsets and mobile platforms will be deployed, and large-scale user studies will assess usability in various situations. These efforts should strengthen the system and support future deployment in stationary and wearable systems.

REFERENCES

[1] H. Cecotti, Y. K. Meena, and G. Prasad, "A multimodal virtual keyboard using eye-tracking and hand gesture detection," in *2018 40th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, 2018, pp. 3330–3333.

[2] M. I. Rusydi, Oktrison, W. Azhar, S. W. Oluwarotimi, and F. Rusydi, "Towards hand gesture-based control of virtual keyboards for effective communication," *IOP Conference Series: Materials Science and Engineering*, vol. 602, no. 1, p. 012030, Aug. 2019.

[3] M. A. Rahim, J. Shin, and M. R. Islam, "Hand gesture recognition-based non-touch character writing system on a virtual keyboard," *Multimedia Tools and Applications*, vol. 79, no. 17, pp. 11 813–11 836, 2020.

[4] M. A. Rahim and J. Shin, "Hand movement activity-based character input system on a virtual keyboard," *Electronics*, vol. 9, no. 5, p. 774, 2020.

[5] R. Matlani, R. Dadlani, S. Dumbre, S. Mishra, and A. Tewari, "Virtual mouse using hand gestures," in *2021 International Conference on Technological Advancements and Innovations (ICTAI)*, 2021, pp. 340–345.

[6] S. Shetty, J. Joseph, J. S. Thomas, M. K. Umesh, and V. E, "Hand gesture recognition system to control keyboard function," *International Journal of Creative Research Thoughts (IJCRT)*, vol. 11, no. 5, pp. b73–b78, May 2023.

[7] A. Anandika, M. I. Rusydi, P. P. Utami, R. Hadelina, and M. Sasaki, "Hand gesture to control virtual keyboard using neural network," *Journal of Information Technology and Computer Engineering (JITCE)*, vol. 7, no. 1, pp. 40–48, Mar. 2023.

[8] B. Mallik, M. A. Rahim, A. S. M. Miah, K. S. Yun, and J. Shin, "Virtual keyboard: A real-time hand gesture recognition-based character input system using LSTM and MediaPipe Holistic," *Computer Systems Science and Engineering*, vol. 48, no. 2, pp. 555–570, 2024.

[9] M. Rahim, J. Shin, M. Abul Hashem, H. Akash, M. Hossain, and A. S. M. Miah, "Advanced touchless HCI: A gesture recognition-based multilingual virtual keyboard for character input," *Journal of Tianjin University Science and Technology*, vol. 57, pp. 1–20, Nov. 2024.

[10] K. Pathak, A. Kalra, S. Patil, A. Joshi, and P. Ahire, "Virtual mouse and keyboard using hand gestures," *International Research Journal of Modernization in Engineering Technology and Science*, vol. 5, no. 11, Nov. 2023.

[11] R. Al King, M. H. Al Sibai, and Y. Z. George, "Design and implementation of a human-computer interaction system using hand tracking and gesture recognition with a built-in camera," in *2025 7th International Congress on Human-Computer Interaction, Optimization and Robotic Applications (ICHORA)*, 2025, pp. 1–10.

[12] C. B. S. Lakshmi, S. S. K, H. P. S. G, and R. C, "AirType: An air-tapping keyboard for augmented reality environment," in *2025 International Conference on Innovative Trends in Information Technology (ICITIIT)*, 2025, pp. 1–5.

[13] M. R. Islam, A. Amin, and A. N. Zereen, "Enhancing Bangla language next word prediction and sentence completion through extended RNN with Bi-LSTM model on N-gram language," in *2024 3rd International Conference on Advancement in Electrical and Electronic Engineering (ICAEEE)*, 2024, pp. 1–6.

[14] S. S. Alif, N. A. Promise, and A. N. Zereen, "A two-stage framework for dynamic two-hand on-screen keyboard-mouse interaction," in *4th International Conference on Human-Machine Interaction (ICHMI)*, New York, NY, USA, 2024, pp. 30–37.

[15] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "MediaPipe: A framework for building perception pipelines," *Computing Research Repository (CoRR)*, vol. abs/1906.08172, 2019.

[16] M. Gerlach and F. Font-Clos, "A standardized project gutenberg corpus for statistical analysis of natural language and quantitative linguistics," *Entropy*, vol. 22, no. 1, p. 126, 2020.

[17] U. Fischer and M. Krüger, "Typesetting bangla script with LuaLaTeX," *TUGboat*, vol. 41, pp. 69–70, Jan. 2020.

[18] D. C. Paul, "Over 11,500 bangla news for NLP," 2023, dataset on Kaggle.